

Inline Variables for the Standard Library

- [Discussion](#)
- [Rationale](#)
- [Resolved Issues](#)
- [Proposed Resolution](#)
- [Acknowledgements](#)

Discussion

This proposal attempts to provide combined wording for the changes requested by the NB comments FI 9 and GB 28 against C++17.

The general approach is to follow the suggested directions described by the issues:

1. **Required change:** Use `inline` variables for *exactly* the constants that are listed by at least one of the two NB comments, which are the following ones:

- `piecewise_construct`
- `allocator_arg`
- `nullopt`
- `[in_place, in_place_type, in_place_index]`: Already part of the working paper.
- `defer_lock`
- `try_to_lock`
- `adopt_lock`
- `bind placeholders _1, _2, ...`
- `ignore`
- `seq`
- `par`
- `par_unseq`

2. **Optional change:** In addition to the changes above introduce *consequently* `inline` variables *for all* namespace-scope variables with internal linkage. **The proper wording for this optional change depends on the resolution of core issue [CWG 1713](#), therefore the proposed wording has an additional sub division!**

The changes in regard to consequently introducing `inline` variables have been made optional, because this step increases the necessary draft changes significantly and that change was not *explicitly* requested by the NB comments.

Rationale

The author of this paper recommends to apply both *two* suggested changes (A) and (B1) inspired by the NB comments as specified below for the following reasons: Inline variables solve a long-standing problem caused by the ODR restrictions of C++. These ODR violations are typically introduced by a basically non-preventable situation where a variable with internal linkage is odr-used in a function with external linkage due to this special constraint imposed (in more precise words as summarized) by 3.2 [basic.def.odr] (6.2.1.3):

-6- There can be more than one definition of a class type (Clause 9), enumeration type (7.2), inline function with

external linkage (7.1.6), inline variable with external linkage (7.1.6), class template (Clause 14), non-static function template (14.5.6), static data member of a class template (14.5.1.3), member function of a class template (14.5.1.1), or template specialization for which some template parameters are not specified (14.7, 14.5.5) in a program provided that each definition appears in a different translation unit, and provided the definitions satisfy the following requirements. Given such an entity named `D` defined in more than one translation unit, then

- (6.1) — [...]
- (6.2) — in each definition of `D`, corresponding names, looked up according to 3.4, shall refer to an entity defined within the definition of `D`, or shall refer to the same entity, after overload resolution (13.3) and after matching of partial template specialization (14.8.3), except that a name can refer to
 - (6.2.1) — a non-volatile `const` object with internal or no linkage if the object
 - (6.2.1.1) — [...]
 - (6.2.1.2) — [...]
 - (6.2.1.3) — is not odr-used in any definition of `D`, and
 - (6.2.1.4) — [...]
 - (6.2.2) — a reference with internal or no linkage initialized with a constant expression such that the reference refers to the same entity in all definitions of `D`;
- (6.3) — [...]

It should be pointed out, that similar changes to constant `static` data members are not needed, because they have external linkage and `constexpr` static data members are already implicitly `inline` variables (7.1.5 [dcl.constexpr] p1).

Another important point to mention is, that as of the current working draft, variable template constants also have internal linkage and are therefore susceptible to the ODR violation problem unless declared as `inline` variables, as specified in 3.5 [basic.link] p3:

- 3- A name having namespace scope (3.3.6) has internal linkage if it is the name of
 - (3.1) — [...]
 - (3.2) — a non-inline variable of non-volatile `const`-qualified type that is neither explicitly declared `extern` nor previously declared to have external linkage; [...]

It is expected that this state will change when [CWG 1713](#) will be resolved.

Resolved Issues

If the proposed resolution would be accepted, the following library issues will be resolved:

Number	Description
2888	Variables of library tag types need to be <code>inline</code> variables
2889	Mark <code>constexpr</code> global variables as <code>inline</code>

Proposed Resolution

At some places below, additional markup of the form

[Drafting notes: whatever — end drafting notes]

is provided, which is *not* part of the normative wording, but is solely shown to provide additional information to the reader about the rationale of the concrete wording.

The below list of suggested changes is split into two halves: List (A) is an enumeration of suggested *minimalistic* wording changes and only shows changes to types *explicitly* listed in either of the NB comments, while list (B1) and list (B2) separately list the *additional* necessary changes when a full introduction of `inline` variables are considered, depending on the outcome of [CWG 1713](#).

The proposed wording changes refer to the Standard C++ working draft [N4640](#).

[Drafting notes: Searching for current usages of global constant variables in the Library, the following locations are *unaffected*:

1. 18.6.1 [new.syn]: nothrow [Constant with external linkage]
2. 20.2.1 [utility.syn] "*in-place construction*": in_place, in_place_type, in_place_index [Already part of the working paper]

— end drafting notes]

A. *Minimalistic* suggestion (only the union of those requested by the NB comments):

1. Change 20.2.1 [utility.syn], header <utility> synopsis, as indicated:

```
inline constexpr piecewise_construct_t piecewise_construct{};
```

2. Change 20.4.5 [pair.piecewise] before p1 as indicated:

```
struct piecewise_construct_t {  
    explicit piecewise_construct_t() = default;  
};  
inline constexpr piecewise_construct_t piecewise_construct{};
```

3. Change 20.5.2 [tuple.syn], header <tuple> synopsis, as indicated:

```
// 20.5.3.4, tuple creation functions  
inline constexpr unspecified ignore;
```

4. Change 20.6.2 [optional.syn], header <optional> synopsis, as indicated:

```
// 20.6.4, no-value state indicator  
struct nullopt_t{see below};  
inline constexpr nullopt_t nullopt(unspecified);
```

5. Change 20.6.4 [optional.nullopt] as indicated:

```
struct nullopt_t{see below};  
inline constexpr nullopt_t nullopt(unspecified);
```

6. Change 20.10.2 [memory.syn], header <memory> synopsis, as indicated:

```
// 20.10.6, allocator argument tag  
struct allocator_arg_t { explicit allocator_arg_t() = default; };  
inline constexpr allocator_arg_t allocator_arg{};
```

7. Change 20.10.6 [allocator.tag] as indicated:

```
namespace std {  
    struct allocator_arg_t { explicit allocator_arg_t() = default; };  
    inline constexpr allocator_arg_t allocator_arg{};  
}
```

8. Change 20.14.11.4 [func.bind.place] as indicated:

-2- Placeholders should be defined as:

```
inline constexpr unspecified _1{};
```

If they are not, they shall be declared as:

```
extern unspecified _1;
```

9. Change 20.19.2 [execution.syn], header <execution> synopsis, as indicated:

```
// 20.19.7, execution policy objects  
inline constexpr sequenced_policy seq{ unspecified };  
inline constexpr parallel_policy par{ unspecified };  
inline constexpr parallel_unsequenced_policy par_unseq{ unspecified };
```

10. Change 20.19.7 [execpol.objects] as indicated:

```
inline constexpr execution::sequenced_policy execution::seq{ unspecified };
inline constexpr execution::parallel_policy execution::par{ unspecified };
inline constexpr execution::parallel_unsequenced_policy execution::par_unseq{ unspecified };
```

11. Change 30.4.1 [mutex.syn], header <mutex> synopsis, as indicated:

```
inline constexpr defer_lock_t defer_lock { };
inline constexpr try_to_lock_t try_to_lock { };
inline constexpr adopt_lock_t adopt_lock { };
```

12. Change 30.4.4 [thread.lock] p2 as indicated:

```
inline constexpr defer_lock_t defer_lock { };
inline constexpr try_to_lock_t try_to_lock { };
inline constexpr adopt_lock_t adopt_lock { };
```

B1. *Additional* suggestion (all remaining cases *excluding variable templates* where inline variables would be appropriate, assumes that [CWG 1713](#) has been resolved):

1. Change 17.4.2.1.3 [enumerated.types] as indicated:

-2- The enumerated type *enumerated* can be written:

```
enum enumerated { V0, V1, V2, V3, ..... };
```

```
inline static constexpr enumerated C0 (V0);
inline static constexpr enumerated C1 (V1);
inline static constexpr enumerated C2 (V2);
inline static constexpr enumerated C3 (V3);
.....
```

2. Change 17.4.2.1.4 [bitmask.types] as indicated:

-2- The bitmask type *bitmask* can be written:

```
[...]
inline constexpr bitmask C0(V0);
inline constexpr bitmask C1(V1);
inline constexpr bitmask C2(V2);
inline constexpr bitmask C3(V3);
```

3. Change 18.6.1 [new.syn], header <new> synopsis, as indicated:

[Drafting notes: The *removal* of the explicit static specifier for the namespace-scope constants hardware_destructive_interference_size and hardware_constructive_interference_size is still *required* because adding inline alone would still not solve the ODR violation problem here. — end drafting notes]

```
[...]
// 18.6.5, hardware interference size
inline static constexpr size_t hardware_destructive_interference_size = implementation-defined;
inline static constexpr size_t hardware_constructive_interference_size = implementation-defined;
```

4. Change 18.6.5 [hardware.interference] as indicated:

```
inline constexpr size_t hardware_destructive_interference_size = implementation-defined;
```

-1- [...]

```
inline constexpr size_t hardware_constructive_interference_size = implementation-defined;
```

-2- [...]

5. Change 20.2.1 [utility.syn], header <utility> synopsis, as indicated:

[Drafting notes: With the acceptance of [CWG 1713](#) the inline specifier is no longer required for const-qualified variable templates, therefore it is recommended to remove them from the two variable templates in_place_type and in_place_index (but not from the non-template constant in_place!) — end drafting notes]

```
template <class T> inline constexpr in_place_type_t<T> in_place_type{};
```

```
[...]
template <size_t I> inline constexpr in_place_index_t<I> in_place_index{};
```

6. Change 20.7.2 [variant.syn], header <variant> synopsis, as indicated:

```
inline constexpr size_t variant_npos = -1;
```

7. Change 28.5.1 [re.synopt], bitmask type syntax_option_type synopsis, as indicated:

```
namespace std::regex_constants {
    using syntax_option_type = T1;
    inline constexpr syntax_option_type icafe = unspecified;
    inline constexpr syntax_option_type nosubs = unspecified;
    inline constexpr syntax_option_type optimize = unspecified;
    inline constexpr syntax_option_type collate = unspecified;
    inline constexpr syntax_option_type ECMAScript = unspecified;
    inline constexpr syntax_option_type basic = unspecified;
    inline constexpr syntax_option_type extended = unspecified;
    inline constexpr syntax_option_type awk = unspecified;
    inline constexpr syntax_option_type grep = unspecified;
    inline constexpr syntax_option_type egrep = unspecified;
    inline constexpr syntax_option_type multiline = unspecified;
}
```

8. Change 28.5.2 [re.matchflag], bitmask type match_flag_type synopsis, as indicated:

```
namespace std::regex_constants {
    using match_flag_type = T2;
    inline constexpr match_flag_type match_default = {};
    inline constexpr match_flag_type match_not_bol = unspecified;
    inline constexpr match_flag_type match_not_eol = unspecified;
    inline constexpr match_flag_type match_not_bow = unspecified;
    inline constexpr match_flag_type match_not_eow = unspecified;
    inline constexpr match_flag_type match_any = unspecified;
    inline constexpr match_flag_type match_not_null = unspecified;
    inline constexpr match_flag_type match_continuous = unspecified;
    inline constexpr match_flag_type match_prev_avail = unspecified;
    inline constexpr match_flag_type format_default = {};
    inline constexpr match_flag_type format_sed = unspecified;
    inline constexpr match_flag_type format_no_copy = unspecified;
    inline constexpr match_flag_type format_first_only = unspecified;
}
```

9. Change 28.5.3 [re.err], error_type synopsis, as indicated:

```
namespace std::regex_constants {
    using error_type = T3;
    inline constexpr error_type error_collate = unspecified;
    inline constexpr error_type error_ctype = unspecified;
    inline constexpr error_type error_escape = unspecified;
    inline constexpr error_type error_backref = unspecified;
    inline constexpr error_type error_brack = unspecified;
    inline constexpr error_type error_paren = unspecified;
    inline constexpr error_type error_brace = unspecified;
    inline constexpr error_type error_badbrace = unspecified;
    inline constexpr error_type error_range = unspecified;
    inline constexpr error_type error_space = unspecified;
    inline constexpr error_type error_badrepeat = unspecified;
    inline constexpr error_type error_complexity = unspecified;
    inline constexpr error_type error_stack = unspecified;
}
```

B2. *Additional suggestion* (all remaining cases *including variable templates* where inline variables would be appropriate, assumes that [CWG 1713](#) has *not* been resolved):

1. Change 17.4.2.1.3 [enumerated.types] as indicated:

-2- The enumerated type *enumerated* can be written:

```
enum enumerated { V0, V1, V2, V3, ..... };

inline static const enumerated C0 (V0);
inline static const enumerated C1 (V1);
inline static const enumerated C2 (V2);
inline static const enumerated C3 (V3);
.....
```

2. Change 17.4.2.1.4 [bitmask.types] as indicated:

-2- The bitmask type *bitmask* can be written:

```
[...]  
inline constexpr bitmask C0(V0);  
inline constexpr bitmask C1(V1);  
inline constexpr bitmask C2(V2);  
inline constexpr bitmask C3(V3);
```

3. Change 18.6.1 [new.syn], header <new> synopsis, as indicated:

[Drafting notes: The *removal* of the explicit static specifier for the namespace-scope constants `hardware_destructive_interference_size` and `hardware_constructive_interference_size` is still *required* because adding `inline` alone would still not solve the ODR violation problem here. — end drafting notes]

```
[...]  
// 18.6.5, hardware interference size  
inline static constexpr size_t hardware_destructive_interference_size = implementation-defined;  
inline static constexpr size_t hardware_constructive_interference_size = implementation-defined;
```

4. Change 18.6.5 [hardware.interference] as indicated:

```
inline constexpr size_t hardware_destructive_interference_size = implementation-defined;  
  
-1- [...]  
  
inline constexpr size_t hardware_constructive_interference_size = implementation-defined;  
  
-2- [...]
```

5. Change 19.5.1 [system_error.syn], header <system_error> synopsis, as indicated:

```
[...]  
// 19.5, system error support  
template <class T> inline constexpr bool is_error_code_enum_v  
    = is_error_code_enum<T>::value;  
template <class T> inline constexpr bool is_error_condition_enum_v  
    = is_error_condition_enum<T>::value;
```

6. Change 20.5.2 [tuple.syn], header <tuple> synopsis, as indicated:

```
[...]  
// 20.5.3.6, tuple helper classes  
template <class T>  
    inline constexpr size_t tuple_size_v = tuple_size<T>::value;
```

7. Change 20.7.2 [variant.syn], header <variant> synopsis, as indicated:

```
template <class T>  
    inline constexpr size_t variant_size_v = variant_size<T>::value;  
  
[...]  
  
inline constexpr size_t variant_npos = -1;
```

8. Change 20.10.2 [memory.syn], header <memory> synopsis, as indicated:

```
// 20.10.7.1, uses_allocator  
template <class T, class Alloc>  
    inline constexpr bool uses_allocator_v = uses_allocator<T, Alloc>::value;
```

9. Change 20.14.1 [functional.syn], header <functional> synopsis, as indicated:

```
// 20.14.11, function object binders  
template <class T>  
    inline constexpr bool is_bind_expression_v = is_bind_expression<T>::value;  
template <class T>  
    inline constexpr int is_placeholder_v = is_placeholder<T>::value;
```

10. Change 20.15.2 [meta.type.synop], header <type_traits> synopsis, as indicated:

```
[...]  
// 20.15.4.1, primary type categories  
template <class T> inline constexpr bool is_void_v  
    = is_void<T>::value;  
template <class T> inline constexpr bool is_null_pointer_v
```

```

    = is_null_pointer<T>::value;
template <class T> inline constexpr bool is_integral_v
    = is_integral<T>::value;
template <class T> inline constexpr bool is_floating_point_v
    = is_floating_point<T>::value;
template <class T> inline constexpr bool is_array_v
    = is_array<T>::value;
template <class T> inline constexpr bool is_pointer_v
    = is_pointer<T>::value;
template <class T> inline constexpr bool is_lvalue_reference_v
    = is_lvalue_reference<T>::value;
template <class T> inline constexpr bool is_rvalue_reference_v
    = is_rvalue_reference<T>::value;
template <class T> inline constexpr bool is_member_object_pointer_v
    = is_member_object_pointer<T>::value;
template <class T> inline constexpr bool is_member_function_pointer_v
    = is_member_function_pointer<T>::value;
template <class T> inline constexpr bool is_enum_v
    = is_enum<T>::value;
template <class T> inline constexpr bool is_union_v
    = is_union<T>::value;
template <class T> inline constexpr bool is_class_v
    = is_class<T>::value;
template <class T> inline constexpr bool is_function_v
    = is_function<T>::value;

```

// 20.15.4.2, composite type categories

```

template <class T> inline constexpr bool is_reference_v
    = is_reference<T>::value;
template <class T> inline constexpr bool is_arithmetic_v
    = is_arithmetic<T>::value;
template <class T> inline constexpr bool is_fundamental_v
    = is_fundamental<T>::value;
template <class T> inline constexpr bool is_object_v
    = is_object<T>::value;
template <class T> inline constexpr bool is_scalar_v
    = is_scalar<T>::value;
template <class T> inline constexpr bool is_compound_v
    = is_compound<T>::value;
template <class T> inline constexpr bool is_member_pointer_v
    = is_member_pointer<T>::value;

```

// 20.15.4.3, type properties

```

template <class T> inline constexpr bool is_const_v
    = is_const<T>::value;
template <class T> inline constexpr bool is_volatile_v
    = is_volatile<T>::value;
template <class T> inline constexpr bool is_trivial_v
    = is_trivial<T>::value;
template <class T> inline constexpr bool is_trivially_copyable_v
    = is_trivially_copyable<T>::value;
template <class T> inline constexpr bool is_standard_layout_v
    = is_standard_layout<T>::value;
template <class T> inline constexpr bool is_pod_v
    = is_pod<T>::value;
template <class T> inline constexpr bool is_empty_v
    = is_empty<T>::value;
template <class T> inline constexpr bool is_polymorphic_v
    = is_polymorphic<T>::value;
template <class T> inline constexpr bool is_abstract_v
    = is_abstract<T>::value;
template <class T> inline constexpr bool is_final_v
    = is_final<T>::value;
template <class T> inline constexpr bool is_signed_v
    = is_signed<T>::value;
template <class T> inline constexpr bool is_unsigned_v
    = is_unsigned<T>::value;
template <class T, class... Args> inline constexpr bool is_constructible_v
    = is_constructible<T, Args...>::value;
template <class T> inline constexpr bool is_default_constructible_v
    = is_default_constructible<T>::value;
template <class T> inline constexpr bool is_copy_constructible_v
    = is_copy_constructible<T>::value;
template <class T> inline constexpr bool is_move_constructible_v
    = is_move_constructible<T>::value;
template <class T, class U> inline constexpr bool is_assignable_v
    = is_assignable<T, U>::value;
template <class T> inline constexpr bool is_copy_assignable_v
    = is_copy_assignable<T>::value;

```



```

template <class T> inline constexpr bool is_move_assignable_v
    = is_move_assignable<T>::value;
template <class T, class U> inline constexpr bool is_swappable_with_v
    = is_swappable_with<T, U>::value;
template <class T> inline constexpr bool is_swappable_v
    = is_swappable<T>::value;
template <class T> inline constexpr bool is_destructible_v
    = is_destructible<T>::value;
template <class T, class... Args> inline constexpr bool is_trivially_constructible_v
    = is_trivially_constructible<T, Args...>::value;
template <class T> inline constexpr bool is_trivially_default_constructible_v
    = is_trivially_default_constructible<T>::value;
template <class T> inline constexpr bool is_trivially_copy_constructible_v
    = is_trivially_copy_constructible<T>::value;
template <class T> inline constexpr bool is_trivially_move_constructible_v
    = is_trivially_move_constructible<T>::value;
template <class T, class U> inline constexpr bool is_trivially_assignable_v
    = is_trivially_assignable<T, U>::value;
template <class T> inline constexpr bool is_trivially_copy_assignable_v
    = is_trivially_copy_assignable<T>::value;
template <class T> inline constexpr bool is_trivially_move_assignable_v
    = is_trivially_move_assignable<T>::value;
template <class T> inline constexpr bool is_trivially_destructible_v
    = is_trivially_destructible<T>::value;
template <class T, class... Args> inline constexpr bool is_nothrow_constructible_v
    = is_nothrow_constructible<T, Args...>::value;
template <class T> inline constexpr bool is_nothrow_default_constructible_v
    = is_nothrow_default_constructible<T>::value;
template <class T> inline constexpr bool is_nothrow_copy_constructible_v
    = is_nothrow_copy_constructible<T>::value;
template <class T> inline constexpr bool is_nothrow_move_constructible_v
    = is_nothrow_move_constructible<T>::value;
template <class T, class U> inline constexpr bool is_nothrow_assignable_v
    = is_nothrow_assignable<T, U>::value;
template <class T> inline constexpr bool is_nothrow_copy_assignable_v
    = is_nothrow_copy_assignable<T>::value;
template <class T> inline constexpr bool is_nothrow_move_assignable_v
    = is_nothrow_move_assignable<T>::value;
template <class T, class U> inline constexpr bool is_nothrow_swappable_with_v
    = is_nothrow_swappable_with<T, U>::value;
template <class T> inline constexpr bool is_nothrow_swappable_v
    = is_nothrow_swappable<T>::value;
template <class T> inline constexpr bool is_nothrow_destructible_v
    = is_nothrow_destructible<T>::value;
template <class T> inline constexpr bool has_virtual_destructor_v
    = has_virtual_destructor<T>::value;
template <class T> inline constexpr bool has_unique_object_representations_v
    = has_unique_object_representations<T>::value;

// 20.15.5, type property queries
template <class T> inline constexpr size_t alignment_of_v
    = alignment_of<T>::value;
template <class T> inline constexpr size_t rank_v
    = rank<T>::value;
template <class T, unsigned I = 0> inline constexpr size_t extent_v
    = extent<T, I>::value;

// 20.15.6, type relations
template <class T, class U> inline constexpr bool is_same_v
    = is_same<T, U>::value;
template <class Base, class Derived> inline constexpr bool is_base_of_v
    = is_base_of<Base, Derived>::value;
template <class From, class To> inline constexpr bool is_convertible_v
    = is_convertible<From, To>::value;
template <class T, class R = void> inline constexpr bool is_callable_v
    = is_callable<T, R>::value;
template <class T, class R = void> inline constexpr bool is_nothrow_callable_v
    = is_nothrow_callable<T, R>::value;

// 20.15.8, logical operator traits
template<class... B> inline constexpr bool conjunction_v = conjunction<B...>::value;
template<class... B> inline constexpr bool disjunction_v = disjunction<B...>::value;
template<class B> inline constexpr bool negation_v = negation<B>::value;

```

11. Change 20.16.2 [ratio.syn], header <ratio> synopsis, as indicated:

```

template <class R1, class R2>
    inline constexpr bool ratio_equal_v = ratio_equal<R1, R2>::value;
template <class R1, class R2>

```



```

    inline constexpr bool ratio_not_equal_v = ratio_not_equal<R1, R2>::value;
template <class R1, class R2>
    inline constexpr bool ratio_less_v = ratio_less<R1, R2>::value;
template <class R1, class R2>
    inline constexpr bool ratio_less_equal_v = ratio_less_equal<R1, R2>::value;
template <class R1, class R2>
    inline constexpr bool ratio_greater_v = ratio_greater<R1, R2>::value;
template <class R1, class R2>
    inline constexpr bool ratio_greater_equal_v = ratio_greater_equal<R1, R2>::value;

```

12. Change 20.17.2 [time.syn], header <chrono> synopsis, as indicated:

```

// 20.17.4, customization traits
template <class Rep> struct treat_as_floating_point;
template <class Rep> struct duration_values;
template <class Rep> inline constexpr bool treat_as_floating_point_v
    = treat_as_floating_point<Rep>::value;

```

13. Change 20.19.2 [execution.syn], header <execution> synopsis, as indicated:

```

namespace std {
    // 20.19.3, execution policy type trait
    template<class T> struct is_execution_policy;
    template<class T> inline constexpr bool is_execution_policy_v = is_execution_policy<T>::value;
}

```

14. Change 28.5.1 [re.synopt], bitmask type syntax_option_type synopsis, as indicated:

```

namespace std::regex_constants {
    using syntax_option_type = T1;
    inline constexpr syntax_option_type icae = unspecified;
    inline constexpr syntax_option_type nosubs = unspecified;
    inline constexpr syntax_option_type optimize = unspecified;
    inline constexpr syntax_option_type collate = unspecified;
    inline constexpr syntax_option_type ECMAScript = unspecified;
    inline constexpr syntax_option_type basic = unspecified;
    inline constexpr syntax_option_type extended = unspecified;
    inline constexpr syntax_option_type awk = unspecified;
    inline constexpr syntax_option_type grep = unspecified;
    inline constexpr syntax_option_type egrep = unspecified;
    inline constexpr syntax_option_type multiline = unspecified;
}

```

15. Change 28.5.2 [re.matchflag], bitmask type match_flag_type synopsis, as indicated:

```

namespace std::regex_constants {
    using match_flag_type = T2;
    inline constexpr match_flag_type match_default = {};
    inline constexpr match_flag_type match_not_bol = unspecified;
    inline constexpr match_flag_type match_not_eol = unspecified;
    inline constexpr match_flag_type match_not_bow = unspecified;
    inline constexpr match_flag_type match_not_eow = unspecified;
    inline constexpr match_flag_type match_any = unspecified;
    inline constexpr match_flag_type match_not_null = unspecified;
    inline constexpr match_flag_type match_continuous = unspecified;
    inline constexpr match_flag_type match_prev_avail = unspecified;
    inline constexpr match_flag_type format_default = {};
    inline constexpr match_flag_type format_sed = unspecified;
    inline constexpr match_flag_type format_no_copy = unspecified;
    inline constexpr match_flag_type format_first_only = unspecified;
}

```

16. Change 28.5.3 [re.err], error_type synopsis, as indicated:

```

namespace std::regex_constants {
    using error_type = T3;
    inline constexpr error_type error_collate = unspecified;
    inline constexpr error_type error_ctype = unspecified;
    inline constexpr error_type error_escape = unspecified;
    inline constexpr error_type error_backref = unspecified;
    inline constexpr error_type error_brack = unspecified;
    inline constexpr error_type error_paren = unspecified;
    inline constexpr error_type error_brace = unspecified;
    inline constexpr error_type error_badbrace = unspecified;
    inline constexpr error_type error_range = unspecified;
    inline constexpr error_type error_space = unspecified;
    inline constexpr error_type error_badrepeat = unspecified;
    inline constexpr error_type error_complexity = unspecified;
}

```

```
    inline constexpr error_type error_stack = unspecified;  
}
```

Acknowledgements

I would like to thank Richard Smith for clarifying core-language relevant matters and for his generous acceptance even of the huge editorial changes that would result when option (B2) would be voted in.